

FORMATION EMBARQUÉE

TD 05 Java

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 05 — Java : énoncés et corrigés détaillés

Corrigés compilables avec un JDK ≥ 17 (`javac` puis `java`). Chaque exercice illustre une brique du rôle de Java autour de l'embarqué : modélisation objet, concurrence, décodage binaire, traitement de données.

Exercice 1 — Hiérarchie de capteurs et journalisation CSV

Énoncé. `Capteur` (abstraite), `CapteurTemp`, `CapteurHum`, et une classe `Station` qui interroge une `List<Capteur>` et journalise en CSV.

Corrigé

```
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.time.Instant;
import java.util.List;

abstract class Capteur {
    private final String nom;
    private final String unite;

    protected Capteur(String nom, String unite) {
        this.nom = nom;
        this.unite = unite;
    }

    public String getNom() { return nom; }
    public String getUnite() { return unite; }

    /** Chaque capteur concret fournit SA façon de mesurer. */
    public abstract double lire();
}

class CapteurTemp extends Capteur {
    CapteurTemp() { super("temperature", "°C"); }
    @Override public double lire() { return 18 + Math.random() * 10; } // simulé
}

class CapteurHum extends Capteur {
    CapteurHum() { super("humidite", "%"); }
    @Override public double lire() { return 30 + Math.random() * 40; }
}

class Station {
    private final List<Capteur> capteurs;
    private final Path fichier;

    Station(List<Capteur> capteurs, Path fichier) {
        this.capteurs = List.copyOf(capteurs); // copie défensive : la liste
        this.fichier = fichier; // de l'appelant peut changer
    }

    /** Une ligne CSV par capteur : horodatage;nom;valeur;unite */
    public void releve() throws IOException {
        try (PrintWriter out = new PrintWriter(Files.newBufferedWriter(
            fichier, StandardOpenOption.CREATE, StandardOpenOption.APPEND))) {
            Instant t = Instant.now();
            for (Capteur c : capteurs) {
                out.printf("%s;%s;%.2f;%s\n", t, c.getNom(), c.lire(), c.getUnite());
            }
        } // try-with-resources : fichier fermé même en cas d'exception
    }
}
```

```

    }
}

public class Main {
    public static void main(String[] args) throws IOException {
        Station station = new Station(
            List.of(new CapteurTemp(), new CapteurHum()),
            Path.of("mesures.csv"));
        for (int i = 0; i < 5; i++) station.releve();
        System.out.println("5 relevés écrits dans mesures.csv");
    }
}

```

Points de correction : le polymorphisme (`Station` ne connaît que `Capteur` , ajouter un capteur = zéro modification de `Station`) ; la copie défensive `List.copyOf` ; le try-with-resources (le RAII de Java, cf. TD 02 exercice 3 — même idée, autre langage).

Exercice 2 — Producteur/consommateur avec `BlockingQueue`

Corrigé

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.TimeUnit;

record Mesure(String capteur, double valeur, long horodatageMs) {}

public class ProdCons {
    public static void main(String[] args) throws InterruptedException {
        // File BORNÉE (64) : si le consommateur décroche, le producteur
        // se bloque au lieu de remplir la RAM – même logique que le ring
        // buffer plein du TD 01.
        BlockingQueue<Mesure> file = new ArrayBlockingQueue<>(64);

        Thread producteur = new Thread(() -> {
            try {
                while (!Thread.currentThread().isInterrupted()) {
                    file.put(new Mesure("temp", 18 + Math.random() * 10,
                                          System.currentTimeMillis()));
                    Thread.sleep(200); // 5 mesures/s
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); // on RESTAURE le drapeau
            }
        }, "acquisition");

        Thread consommateur = new Thread(() -> {
            try {
                while (!Thread.currentThread().isInterrupted()) {
                    // poll avec timeout plutôt que take() : permet de tester
                    // régulièrement le drapeau d'interruption
                    Mesure m = file.poll(500, TimeUnit.MILLISECONDS);
                    if (m != null)
                        System.out.printf("traite : %s = %.1f%n", m.capteur(), m.valeur());
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "traitement");

        producteur.start();
        consommateur.start();

        Thread.sleep(3000); // le programme tourne 3 s...
        producteur.interrupt(); // ...puis arrêt PROPRE des deux threads
        consommateur.interrupt();
        producteur.join();
        consommateur.join();
        System.out.println("arret propre, file restante = " + file.size());
    }
}
```

```
}  
}
```

Points de correction : - `BlockingQueue` remplace mutex + condition + tampon écrits à la main : toute la synchronisation est dans la structure. - **Arrêt propre :** `interrupt()` + `join()` , et le `catch` qui restaure le drapeau. Un thread tué brutalement (il n'y a d'ailleurs pas de moyen sûr de le faire en Java) laisse des ressources en vrac — même souci qu'un firmware coupé en pleine écriture flash. - La file **bornée** est un choix de conception, pas un détail (contre- pression / backpressure).

Exercice 3 — Décodeur de trame binaire (le piège des `byte` signés)

Énoncé. `byte[] {0xA5, id, hi, lo}` → objet `Mesure` , avec validation.

Corrigé

```
import java.util.Optional;

record TrameMesure(int id, int valeur) {}

class DecodeurTrame {
    static final int TETE = 0xA5;
    static final int LONGUEUR = 4;

    /** Optional plutôt que null ou exception : trame invalide = cas NORMAL
     *  sur un bus réel, pas un cas exceptionnel. */
    static Optional<TrameMesure> decoder(byte[] trame) {
        if (trame == null || trame.length != LONGUEUR)
            return Optional.empty();

        // PIÈGE CENTRAL : en Java, byte est SIGNÉ (-128..127).
        // (byte)0xA5 vaut -91 ! Le masque & 0xFF "remonte" en int non signé.
        if ((trame[0] & 0xFF) != TETE)
            return Optional.empty();

        int id = trame[1] & 0xFF;
        int valeur = ((trame[2] & 0xFF) << 8) | (trame[3] & 0xFF);
        return Optional.of(new TrameMesure(id, valeur));
    }
}

public class TestDecodeur {
    public static void main(String[] args) {
        byte[] ok = {(byte) 0xA5, 0x07, (byte) 0x01, (byte) 0xF4}; // valeur 500
        byte[] ko = {(byte) 0x55, 0x07, 0x00, 0x00}; // mauvaise tête

        System.out.println(DecodeurTrame.decoder(ok)); // Optional[TrameMesure[id=7, valeur=500]
        System.out.println(DecodeurTrame.decoder(ko)); // Optional.empty
        System.out.println(DecodeurTrame.decoder(new byte[2])); // Optional.empty
    }
}
```

LE point de correction : sans `& 0xFF`, `trame[0] != TETE` compare -91 à 165 → toute trame valide est rejetée. C'est le bug Java n°1 en manipulation binaire (sockets, ports série, fichiers). Compare avec le C (TD 01 ex. 4) : mêmes vérifications, mêmes décalages — seule la gestion du signe diffère.

Exercice 4 — Statistiques d'un CSV avec les streams

Énoncé. Lire un CSV `horodatage;capteur;valeur`, sortir min/max/moyenne par capteur.

Corrigé

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.DoubleSummaryStatistics;
import java.util.Map;
import java.util.stream.Collectors;

public class StatsCsv {
    public static void main(String[] args) throws IOException {
        Map<String, DoubleSummaryStatistics> stats;
        try (var lignes = Files.lines(Path.of("mesures.csv"))) {
            stats = lignes
                .map(l -> l.split(";"))
                .filter(c -> c.length >= 3) // ligne malformée → ignorée
                .collect(Collectors.groupingBy(
                    c -> c[1], // clé : nom du capteur
                    Collectors.summarizingDouble(c -> {
                        try { return Double.parseDouble(c[2]); }
                        catch (NumberFormatException e) { return Double.NaN; }
                    }
                ));
        }

        stats.forEach((capteur, s) ->
            System.out.printf("%-12s : n=%d min=%.2f max=%.2f moy=%.2f%n",
                capteur, s.getCount(), s.getMin(), s.getMax(), s.getAverage()));
    }
}
```

Points de correction : `Files.lines` dans un try-with-resources (c'est un flux paresseux qui tient le fichier ouvert) ; les lignes malformées sont **filtrées, pas fatales** ; `DoubleSummaryStatistics` donne min/max/moyenne en une seule passe — pas trois parcours du fichier.

Exercice 5 (complément) — Question de conception

Énoncé. Ton ESP32 envoie une mesure JSON par seconde en MQTT. Tu dois : l'historiser, afficher la dernière valeur sur une page web, alerter au-delà d'un seuil. Découpe une application Java en classes/threads et justifie.

Corrigé (éléments attendus)

- **Un thread d'ingestion** : le callback MQTT ne fait que désérialiser et poser la mesure dans une `BlockingQueue` (règle « ISR courte », version serveur : un callback de bibliothèque réseau ne doit jamais bloquer).
- **Un thread de traitement** : consomme la file → écrit en base (SQLite/ PostgreSQL), met à jour un `volatile Mesure derniere` (ou `AtomicReference`), évalue le seuil.
- **Alerte avec hystérésis et anti-rafale** : même logique qu'au TD 03 — on n'envoie pas 60 e-mails/minute quand la valeur oscille autour du seuil.

- **Serveur HTTP** dans son propre pool de threads, qui ne lit que des données déjà prêtes (jamais la base sur le chemin chaud).
 - Séparation en couches : **transport** (MQTT), **domaine** (Mesure, règles d'alerte — **testables sans réseau**), **persistance**, **web**. La logique métier ne dépend d'aucune bibliothèque : c'est le même principe que l'interface **IAfficheur** du TD 02.
-

Auto-évaluation

Sans notes : expliquer `& 0xFF` sur un `byte` ; try-with-resources vs RAII C++ ; pourquoi une file bornée ; le protocole d'arrêt propre d'un thread ; et citer deux endroits où Java touche l'industriel (Modbus TCP, MQTT, Android, SCADA type Ignition).